



JavaScript Security: Best Practices

Course Overview

Course Overview

Hi everyone. My name is Marcin Hoppe, and welcome to my Pluralsight course, JavaScript Security: Best Practices. I am a software engineer specialized in information security. I am also a member of the Node.js Ecosystem Security Working Group and an active contributor to the Open Source Security Foundation. The web is a wonderful, but dangerous place, Attackers, security vulnerabilities, and data breaches are a fact of life for many online businesses. JavaScript has a very special place in the web ecosystem. JavaScript code can be a target of an attack, but it can also be a tool that hackers use to breach our applications. In this course, we're going to learn how to improve the security of your JavaScript code. We are going to cover the JavaScript security model, dynamic type system vulnerabilities, code injection attacks, prototype pollution, and JavaScript security testing tools. By the end of this course, you'll know how to find, fix, and prevent the most common JavaScript security bugs. Before beginning the course, you should be familiar with JavaScript programming concepts such as variables, types, objects, and functions. I hope you'll join me on this journey to learn how to write more secure code with the JavaScript Security: Best Practices course, at Pluralsight.

Understanding JavaScript Security

Introduction

Welcome to the JavaScript Security: Best Practices course, here on Pluralsight. My name is Marcin Hoppe, and in this course, I will teach you how to write more secure and robust JavaScript code. The web runs on JavaScript. It is the dominant programming language for writing browser applications, and thanks to the Node.js runtime, it is increasingly common to see it in the back end too. The quality of JavaScript code is crucial for security of web applications. This course, however, is not about general web application security. We will not address problems that can affect applications written in any programming language. In this course, we focus on security issues that are unique to JavaScript, and they are a result of its dynamic nature. I will teach you how to identify such vulnerabilities, how to fix them, and prevent those issues from creeping into your code. In this module, we will focus on the fundamental role that JavaScript plays in web application security. JavaScript can contain



vulnerabilities, but in some cases it may even become an attack vector. There are two popular environments for running JavaScript code, and both of them have very different security properties. First, we will take a look at how browsers run JavaScript, and then we will see how Node.js is different. Then, we will look at language features that may lead to security vulnerabilities, dynamic typing, dynamic code execution, and prototypal inheritance. We will wrap this module up with an example of a simple coding mistake, literally just a missing character, that leads to a significant leak of sensitive data. Information security professionals are well known for specific jargon to use. We will not use it here, but it is important to understand some basic concepts of web security. Attacks against web applications are carried out by people. You may have an image of a person in a black hoodie typing at their keyboard in their basement, but the reality is much more nuanced. Attackers differ based on their capabilities and motivations. They can be teenagers wanting to impress their friends, fired employees seeking revenge, as well as criminals breaking into applications for money. Attacks would not be possible without vulnerabilities. Vulnerabilities are technical flaws in the system that allow people with malicious intent to break into our applications and systems. They can be simple bugs in the code, fundamental architecture flaws or configuration mistakes. All of them can lead to data breaches. Those that usually hit the headlines are about leaking millions and millions of sensitive data records, such as credit card numbers. Data breaches can also involve abusing application functionality, for example toward their goods without paying or getting a refund for goods that were never purchased in the first place. The most common web application architecture has three tiers, the browser, the server, and the database JavaScript code can run both in the browser and the user's device, such as a laptop or smartphone, or on the server using Node.js. Vulnerabilities in server-side code may allow attackers to breach access to the application datastore. A successful attack in a database may lead to a data breach that involves many users. The impact of a vulnerability in client-side code is typically limited to a single user. That sounds like good news. Unfortunately, bugs in JavaScript code running in a browser may allow attackers to impersonate the victim and to perform actions on their behalf. In this case, the vulnerable JavaScript code becomes an attack vector.

How Browser Execute JavaScript Code

JavaScript was created to add interactivity to HTML pages. Web browsers are the native environment to run JavaScript code. In fact, JavaScript is the dominant programming language in this space. When the user visits a web page, the browser downloads the HTML code of that page, as well as all the other assets needed to display this page. This includes CSS style sheets, images, and JavaScript code. Browsers allow users to visit multiple pages at the same time in tabs or separate browser windows. This means that at any given time, JavaScript code downloaded from several different sites is executed in the same browser. If



one of those sites is infected or even owned by the attacker, aren't we at risk of malicious code stealing our data from legitimate sites? Luckily, browsers do not allow for this, and every website executes JavaScript code in its own sandbox within the browser. Code from one website cannot access data or functionality from another website. This is one of the most fundamental security properties of the web. Some browsers use very sophisticated sandboxing mechanisms, like running each tab in a separate operating system process. Downloading code over the Secure HTTP Protocol and using Subresource Integrity, or SRI for short, prevents attackers from injecting their own malicious code into benign sites. JavaScript code running in the browser is restricted in what it can do. It has no access to local resources in a user's computer, and this applies to devices such as webcams or microphones, the file system, and the local network. The code can use those resources only using very limited browser APIs. This allows the browser to minimize the attack surface and ask the user for explicit consent for using those resources. Code originating from different sites cannot access each other's data and functionality. This allows for even stronger protection of data and code execution within the browser.

How Node.js Executes JavaScript Code

Node.js is a runtime environment for JavaScript based on the V8 engine built for the Google Chrome browser. The unusual thing about it is that it allows JavaScript code to run outside of the browser. It has gained a lot of popularity and has proven to be a popular tool to build command line programs and web applications. It is quite different from the browser from a security perspective. Browsers download the code, and Node.js loads the code from local files, much like other popular programming languages. The permissions model is also different. Browsers treat the code as untrusted and restrict capabilities it has access to, and Node.js treats the code with full trust and grants access to all the privileges the operating system user has access to, including devices, files, and the local network. Attacks based on a security vulnerability in a browser may affect one victim at a time. Bugs in Node.js may allow for full server compromise, potentially leading to a serious data breach.

JavaScript Security Pitfalls

JavaScript is a little bit of an unusual programming language. Its rapid development and massive popularity gave us several language features and coding patterns that may easily lead to exploitable security bugs. JavaScript variables can refer to objects of different types. In statically typed programming languages, variables can only store or reference values of a particular type. An integer can only store numbers, never strings or objects. In JavaScript, a variable can refer to a number, a string or an object, depending on the flow of



control. When you look at the code, you do not always know the types of variables. It may lead to unintentional information disclosure or other security bugs. JavaScript programs can invoke JavaScript engine at runtime. It sounds like a really powerful feature, and it is. It allows for easy processing of complex data formats, such as mathematical formulas or implementing applications that users can extend with their own JavaScript code. Unfortunately, this is also what attackers dream about, the ability to inject their code into your application. JavaScript has a pretty unusual inheritance mechanism. Most mainstream programming languages use classes to express static, hierarchical relationships between types of objects. In JavaScript, the same goal is achieved by building dynamic relationships between individual objects. Each object has a parent object, the prototype it inherits properties from. If attackers can modify the objects forming the prototype chain, they may alter the behavior of your code in unforeseen ways. The dynamism of JavaScript is powerful and flexible. It facilitates rapid development and unlocks programmer productivity. If the same dynamism is abused by attackers, it can lead to security vulnerabilities.

Sample Application

Now let's take a look at the sample application implemented in JavaScript. It is a part of an ecommerce system for specialty coffee lovers. In a true startup fashion, it was built fast and, truth be told, some things might be broken. The development team thinks there might be a few security issues here and there. The first area of concern is the login screen. It looks simple at a glance, but it has some smarts in how it tracks where the user was coming from before they logged in. We will take a closer look at that functionality. The second area that needs attention is user profile management. The implementation uses several programming idioms that depend heavily on the dynamic nature of JavaScript. We will inspect the code of both of those features throughout the course. We start on the home screen of the list of coffee means sold by Wired Brain Coffee. The first functionality is the login screen. We type in the email, and we are in. Notice that now the user's email is displayed in the top right corner and the Logout button is visible. Now that we are logged in, we can also access the user profile management screen. It is quite simple, and it allows us to edit the shipping address. The Email field is read only. Let's log out, and let's go to the profile management screen again. It looks like we first need to log in before we can edit the profile. Notice that now we are taken back to the login screen that has the return URL stored in a JSON object in a query string. User session is implemented using a simple cookie that contains user email. We can easily display it using the browser development tools. Now, let's take a closer look at how this functionality is implemented.

Code Walkthrough



The Wired Brain Coffee ecommerce application consists of two components, server and client. The back-end code is implemented in JavaScript using Node.js and the Express framework. The details of this framework are not relevant for this course, but it allows us to easily handle serving JavaScript code, as well as CSS and HTML assets. It also allows us to dynamically generate and serve JSON documents. All those files are served by the server to the browser. Let's take a look at the server code. The `app.js` file configures the Express framework to serve static files, parse JSON documents, and handle HTML forms. The code in `login.js` handles the login form. It first reads the email and password from the form and checks if they match the users in our database. This is a sample application, so storing user information in a flat JSON file is good enough. If the user credentials are verified, the code sends the cookie back to the browser and redirects the user to the return URL from the JSON object from the query string. If the credentials are not verified, the code returns an HTTP 401 code. The code in `logout.js` is very simple. It removes the cookie and redirects the user back to the home page. `Profile.js` has two functions. One of them is responsible for reading profile information from the JSON file based on some search criteria. The criteria are sent in the field and value query string parameters. Then, we filter the user database based on those criteria. The results are sent back to the browser in a JSON file. The function responsible for saving the profile information is a bit more complex. First, we find a user based on the Email field. If the user has been found, we clone the field from the request using a JavaScript idiom that merges the request object with an empty object. Then, we assign all the fields from this copy to the user object in our user database. Then, we return to user profile and JSON file. I'm sure you have noticed a few helper functions defined in `utils.js`. The filter function returns those elements of the items array where a given field matches the provided value. The `getParams` function retrieves an array of values from the query string object. The values are retrieved by names specified in the params array. The function also gracefully handles missing values returning null, which is a JavaScript value to denote missing data. The merge function performs a deep recursive merge of properties of the source object with the properties of the target object. Using such a function with an empty object as the target is a popular JavaScript idiom for performing deep copies of objects.

Loose Comparison Vulnerability

JavaScript has a dynamic type system. It means that variables can reference values of different types throughout their lifetime. At one point, a variable may refer to a number, and it may refer to a string later on. The rules that describe how operations are applying to values of different types are quite complex and can lead to security issues. When JavaScript code attempts to perform an operation on two values of different types, it needs to convert them to a common type where that operation is well defined. For example, adding a number to a string will convert that number to a string, and the addition operation will



become a concatenation of two strings. This may lead to unexpected code being called if types of variables are not properly tracked and controlled. JavaScript has two comparison operators, strict, also known as triple equals (===), and loose, known as double equals (==). The strict comparison operator compares both the value and type, so a string can never be equal to a number. The loose comparison, when applied to parameters of different types, automatically converts the operands to a common type to make the comparison possible. When using this operator, a string can, in some cases, be equal to a number. Comparing values such as null and undefined is another corner case. Strict comparison will always treat those two values as different, but the loose comparison will treat them as equal. This may lead to security checks being bypassed. If they use the loose comparison, variable types are not enforced. Another aspect of JavaScript dynamism is that it used to be very forgiving of programming errors. Over time, it became clear that certain language features, like giving any function an ability to define a global variable, are dangerous for security and the correctness of the code. Newer versions of JavaScript allow the code to run in so-called strict mode that prohibits some of the problematic behaviors. Strict mode is enabled by putting the use strict string literal at the beginning of a script or function, and it should always be used when writing JavaScript code. Let's get back to the code of the Wired Brain Coffee application. Let's quickly review all the code files for use of the loose comparison operator. If we find it is used in a security check that operates on untrusted input data, it might be a security vulnerability. It looks like the double equals operator is used in the filter utility function and is called from the readProfile function. Now let's take a quick look at how to exploit this bug.

Exploiting the Vulnerability

We identified a potential vulnerability. Now we need a way to make use of this bug to either steal sensitive data or abuse functionality of our sample application. There are a few ways to make use of automatic conversions and loose comparison to invoke an unwanted or unexpected behavior. We can try to use numbers instead of strings, we can use arrays where objects are expected, we can also remove properties that get the JavaScript value undefined. When we have identified input that may potentially lead to an attack, we should work backwards from the vulnerable code to the input data to try to find a way to deliver it to the vulnerable piece of code. When we have that, we can inspect the HTTP requests and responses using browser developer tools or proxies such as Fiddler. The next step is to modify legitimate requests to include malicious data. Finally, we send such modified requests to the application. Let's see how we may do this for the loose comparison we found in the filter function. The loose comparison tries to compare a field of an object, provided as an input parameter, to a specific value, also provided as an input parameter. Both the field and the value are taken from request parameters in the readProfile function. If we could



trick the filter condition to always be true, we would be able to retrieve information about all the users in the database. If the field did not exist, the property retrieval, the bracket notation, would return undefined. If the value was null, the loose comparison would always be true. Luckily for the attacker, it will be null if it is missing from the query string thanks to the `getParams` function. Let's see how we can do that. The first step is to capture the legitimate request. Let's open the browser developer tools and let's inspect the network traffic. The request has a field name and value in the query string. The response contains an array with a single user object matching the provided email value. Now, let's modify the request. Let's remove the value from the query string, and let's change the field name to an invalid value. It's time to send such a modified request to the application. The response now contains information about all the users. Our attack has been successful. Now, let's see how to fix it

Fixing the Code

There are several ways to fix security vulnerabilities coming from the JavaScript dynamic typed system. Using strict mode is a no-brainer, except for cases where backwards compatibility is a concern. All code should run in strict mode. Using loose comparison can lead to strange bugs, and when used in security checks it can lead to their bypass. The strict comparison operator, or triple equals (`===`), should be used instead. Object equality can also be checked using the `Object.is` method, which works almost like triple equals, except for a few corner cases related to numbers. Untrusted input should always be subject to data type verification. All data coming from outside of the application might have potentially been tampered with by the attacker and may cause unexpected type conversions and unwanted behavior. Now, let's fix our vulnerability. The simplest way to fix the loose comparison vulnerability is to replace it with the triple equals operator. Now, let's see if this stopped the attack. Let's send the malicious request again. We need to remove the value parameter and change the field parameter to an invalid value. Now we get the expected result, an empty array. The array is empty because the comparison operator correctly distinguished between null and undefined. If this behavior still seems a little off to you, you're right, we're still relying on arcane rules of comparison of null and undefined values. The input data validation should be a lot stricter and reject incorrectly formed requests. Let's add validation for the field and value parameters and return an HTTP 400 code to inform the caller that the request was malformed. We need to check if the type of both field and value variables is a string. If it is not, we have to return the error code and skip the rest of the `readProfile` function. Let's send the malicious request again. Now, the server properly responds to a malicious attack. We tightened our defenses by verifying that all required parameters are indeed present and that the types are what we expect



Summary

In this module, we covered which JavaScript features may lead to security bugs. We have seen how the dynamic type system may lead to disclosure of sensitive information. In the next two modules, we will dive deep into dynamic code execution and prototypal inheritance, and we will see how these features might be exploited. We have also taken a look at the two most popular environments for running JavaScript code, and we have seen how different they are from a security perspective. Browsers run JavaScript in a secure sandbox, but bugs may be used as an attack vector. JavaScript code running on Node.js has all the privileges of the operating system account it runs under, and the bugs may lead to a full server compromise. You will see one such vulnerability in the next module.

Preventing Code Injection Attacks

Overview

Welcome back to the JavaScript Security: Best Practices course on Pluralsight. My name is Marcin Hoppe, and in this course I will teach you how to write more secure and robust JavaScript code. In the previous module, we introduced the basic notions of web security and explained the role JavaScript code plays in preventing attacks. We also demonstrated how the JavaScript dynamic type system may lead to sensitive data leaks. In this module, we will focus on code injection vulnerabilities. We will see how to find them in our code, we will demonstrate how they allow attackers to execute arbitrary code within our applications, and lastly, we will learn how to fix those types of bugs. JavaScript programs can generate and execute code on the fly. This capability is often called dynamic code execution. If this code is constructed based on input data, and if attackers can tamper with this data, they may find a way to get their code executed. JavaScript has several functions that execute code as a string parameter and then execute it. We will learn what those functions are and what are the differences between them. Each code injection attack is different. They can have a variety of negative impacts, from crashing the application to cause an outage, all the way up to hijacking the application, or even the entire server. We will also discuss coding principles and patterns that prevent code injection vulnerabilities from creeping into our programs and libraries. Let's get back to our three-tier application. The browser sends HTTP requests to the server, the server reads or writes the data from the database, and sends the response back to the browser. Code injection attacks originate from the browser or other user agent. The attacker sends a malicious payload with code to be executed, encoded as data. The application running on the server parses this data. If there is a remote code execution vulnerability, the application



executes the malicious code provided by the attacker. This may allow the attacker to steal sensitive data that only the application is allowed to access

Dynamic Code Execution

JavaScript engines are very versatile. They can run code loaded from a variety of sources. Files downloaded from the web and read from files on disk are the most popular, but the code may also come from user input or program variables. The first phase of running JavaScript is parsing. The JavaScript engine reads the source code, analyzes it, and builds an abstract syntax tree presentation of the script or module. The next phase is compilation. This phase takes the abstract syntax tree as input and translates it to an intermediate representation called bytecode. The final phase is execution. This is where the intermediate representation is processed by the JavaScript engine and the bytecode instructions are executed by the processor. The parse, compile, execute pipeline is invoked when the code is loaded, but it can also be invoked at runtime. This gives JavaScript code an ability to create new code on the fly. Let's take a look at the short program that evaluates arithmetic expressions. The expression variable is a string that stores the expression. This particular body is hard coded, but it could come from an HTML form or a command line parameter. The eval function parses, compiles, and evaluates the JavaScript expression from the variable. We will learn more about the eval function and its properties in the next clip. The result of evaluating the expression is stored in the result variable. The value of this variable is 4. To calculate this value, the JavaScript engine had to parse, compile, and interpret the arithmetic expression as if it was JavaScript code. The type of the result variable is a number, the JavaScript type used to represent integer and floating point values. Using the eval function allowed us to leverage arithmetic expression evaluator built into the JavaScript engine. An alternative solution would be to write the parsing and evaluation code ourselves. Now, let's take a look at which JavaScript constructs allow for dynamic code execution.

Unsafe Functions

JavaScript has two unsafe functions that execute code provided in a variable, eval and the function constructor. The use of eval is straightforward. We call the function and provide the code to be executed as an argument. The function constructor creates a new function. It is a constructor function and should be invoked with the new operator. It takes one or more parameters. The last argument should contain the body of the function, and the preceding arguments should contain names of parameters of the created function. In this example here, variable f refers to a function with one parameter named param and with the body stored in the code variable. This function body can use the param parameter. To invoke f, we need to pass a single argument. There are two ways to



invoke the eval function. Direct invocation is the simplest form shown on the slide. Indirect invocation is any other form that does not look like that, but has the same effect. For example, we could create a variable to be an alias of the eval function and call that alias. This would be an indirect invocation. Dynamic code execution through the function constructor is much simpler. We invoke it like a function through the object returned by the new operator. There are very subtle differences between those two mechanisms. Direct invocation of the eval function gives the executed code access to the current scope, including local variables. This may lead to leaking sensitive data and is not advisable.

Both indirect invocation of eval and using the function constructor only give access to the global scope. If you have to use either of the two unsafe functions, make sure you're using the form that only allows access to the global scope. Eval and the function constructor are not the only popular JavaScript functions that take code in a parameter and execute it. SetTimeout executes provided code with a delay. It has a safe variant where you need to provide a normal JavaScript function as a parameter. It also has an unsafe variant that accepts JavaScript code passed as a string value. In this way, it is equivalent to calling the eval function, but with a delay. setInterval is very similar. It executes the provided code over and over with a delay between invocations. It also has a safe and unsafe variant, just like setTimeout. It is advised to always use the safe variants of both functions.

Finding Unsafe Code

Let's take a look at the process of turning unsafe functions, such as eval, into attacks against web applications. After having found where unsafe functions are used in our source code, we need to take a look at what data can be used to deliver the code of our choosing to those functions. The first step is to track all input data from sources, such as HTML forms, cookies or HTTP requests parameters, to unsafe function calls. Such unsafe function calls are often called sinks. In a well-written application, untrusted input will rarely be passed directly to the sink unmodified. The task of tracking how input data is moved throughout the code is often called taint analysis, or taint tracking. We also need to keep track of all the transformation supply to the input data along the way. Web technologies and protocols often encode data in multiple ways to make them suitable for use in a particular context. Examples of such transformations are a URL encoding and escaping special HTML characters. Such data transformations are often reversible, and we will need to keep that in mind when building malicious payloads. When we have a ready payload to inject code to be executed in a web application, we can follow the three-step process of delivering it that we used in the previous module. First, we inspect the legitimate HTTP traffic using browser developer tools or a proxy. Then, we modify the request to inject our malicious payload. And finally, we deliver the payload to the application. Code injection attacks are one example of a broader category of vulnerabilities.



Any untrusted input that is interpreted without proper validation or sanitization may lead to an exploitable vulnerability. This problem is especially prevalent for database query engines and operating system shells. In this module, we focus on injecting code to JavaScript engines, but the same principles apply for other types of injection attacks. Now, let's get back to the source code of the Wired Brain Coffee ecommerce application. Let's search for the use of unsafe functions. Let's type `eval`, and let's make sure to match the whole words only. This reduces the number of false positives in the search results. We found one use of `eval` in the login form handler. The same procedure should be repeated for other unsafe functions, but luckily they're not used in the Wired Brain Coffee ecommerce system. The `eval` function is used to parse the JSON document passed in the `returnTo` query parameter. The URL field from this JSON document is used later to redirect the user back to the page that they were visiting before the login page. The query string portion of the URL can easily be manipulated by the attacker. In the next clip, we will use it to launch a code injection attack.

Exploiting the Vulnerability

We will perform two code injection attacks against the application using the return URL parameter we discovered in the previous clip. We will use the familiar attack technique based on hijacking legitimate HTTP requests, modifying them to inject malicious payloads, and delivering those payloads to the application. Our first attack will crash the server application, causing a denial-of-service attack. The second attack will be based on the fact that direct invocation of `eval` has access to the current scope and allows the application objects and data to be easily manipulated. Let's log in to the application. Now, let's inspect the POST request, and let's open it for editing. The content of the `returnTo` parameter is directly passed to the `eval` function. Let's try to pass a simple expression that uses the global `process` object and its `exit` method to crash the application. Notice that the browser did not receive any response. A quick look at the application log indeed shows that the process was terminated with the exit code 99 that we passed to the `exit` method. Let's restart the application, and let's take a look at the vulnerable code again. Perhaps there is a more interesting attack we can launch. The called `eval` is a direct invocation. This means that the injected code has access to the current scope, including all the local variables. The `users` variable contains the entire user database. The `res` object from the Express framework allows us to manipulate the content that is sent back to the attacker. Let's get back to the browser. Let's try to inject the code that leaks the user database in a JSON response. Let's inject the code into the `returnTo` parameter, and let's send it again. Now, let's take a look at the response. It's a JSON array with information about all the users of the application. Our attack has been successful. The injected code can also access all the built-in Node.js modules. For example, it can load the `fs` module that allows



the attacker to read arbitrary files from the disk and send them back to the attacker. This can lead to a serious data breach.

Impact of Code Injection Attacks

The impact of code injection attacks can be very severe. It depends on how much the attacker knows about the application and its infrastructure. One of the simplest attacks is denial of service. Crashing the application and removing its files from the disk to prevent it from restarting are all easy to do if the application process has sufficient privileges. Such attacks are usually quite easy to detect and prevent. Modification of application logic is much more difficult to discover and stop. A determined attacker with a knowledge of application code can modify application state to bypass access controls or compromise internal data integrity rules. Sensitive data leaks are a serious concern if an injection vulnerability allows for data to be returned to the attacker. Attackers do not always focus on the application itself. Remote code execution vulnerabilities are often used to establish initial access to the infrastructure. This allows further attacks to be launched via the compromised web application. Web shells allow attackers to execute arbitrary commands on the system that runs the compromised web application. The attacker sends a web shell payload to the application. If the code injection attack is successful, the compromised server starts listening to commands over the HTTP protocol. The attacker can use the web shell to scan the network or escalate access to the server. Code injection attacks against server-side JavaScript applications running on Node.js make it easy to create a web shell. Injected code can easily use the built-in HTTP and OS modules to listen and execute arbitrary commands. Let's take a look at a simple JavaScript web shell. It starts an HTTP server listening on port 8000. The shell commands are sending the `cmd` query string parameter and are immediately executed using the `child_process` module. We will use the minified version of the web shell payload. We are back at the browser. Let's edit our last login request and let's paste the minified version of the web shell. Now, let's open up a new browser tab. Our web shell is listening on port 8000. Let's pass the UNIX `ls` command in the `cmd` parameter, and let's send the request. We can see that the server responded with a list of files in the root folder of the application. Web shells are a powerful tool for attackers to gain information about the compromised system and to launch further attacks.

Fixing the Code

There are several coding pattern principles that minimize the chance of introducing code injection attacks to your code. The first is to avoid using unsafe functions at all cost. If there is no way to dynamically execute JavaScript code, there is no way to abuse this functionality. If using `eval` or similar function is



inevitable, we should validate the input data very carefully. Our initial arithmetic expression sample is a good example. Input validation should ensure that only digits, parentheses, and arithmetic operators are allowed. We should prefer allow lists that explicitly define what input is legitimate over block lists that only define known bad values. Relying on a single line of defense is a risky strategy, and input validation should always be supplemented with sanitization of data passed to outside interpreters to prevent data being treated as code. We should also plan for our defenses to fail. Applying the principle of least authority means that our code runs with the bare minimum of privileges necessary to do the job. The less our application is capable of doing, the smaller the impact in case attackers run a successful code injection attack. Let's take one more look at the vulnerable code in the login.js file. Let's fix the vulnerability by replacing the use of the eval function with a proper JSON parser. JavaScript has a built-in JSON parser available through the global JSON object. Let's pass the returnUrl query string parameter directly to the parse method of that object. Let's perform the log in sequence again. Let's find a POST request that we previously found to be vulnerable to code injection, and let's try to perform our initial denial-of-service attack. Let's pass a call to process.exit to the returnUrl query string parameter, and let's send the request once again. The attack has been stopped and the injected code is not being executed. The application returns an internal error. This may also lead to denial-of-service attacks, and we need to implement better input validation in the login handler. Let's improve the input validation. We need to verify that the returnUrl parameter is valid JSON and that it has a URL field. The easiest way to do it is to wrap the JSON parsing code in a try/catch block. If the parser throws or the URL field is malformed, we need to clear the cookie to make sure we do not allow the user to sign in successfully during an attempted attack. We should also return the HTTP 400 code to indicate that the request was invalid. We can do this by throwing our own error.

Unsafe Code in Third-party Libraries

JavaScript applications are often built on top of dozens, if not hundreds, of open source libraries and frameworks. The npm has become the standard package manager for JavaScript applications, both running in the browser and in Node.js. Many of those third-party packages did not go through a stringent security review and may contain vulnerabilities. Code injection vulnerabilities are not an exception here, and third-party packages may be affected by them, just like your own code. To avoid potential code injection attacks, you should validate input data before passing it to external libraries; otherwise, you are at risk of accidentally opening your application to remote code execution attacks. For certain packages, such as web frameworks, it is not practical or even possible to perform input validation. In such cases, it is a good idea to audit the source code of those libraries for the use of unsafe functions as a precaution. In module 5, we will discuss how to automate such audits using static analysis tools. Math.js is a



mathematical library for JavaScript and Node.js that, among other features, implements the evaluation of complex mathematical expressions. Version 3.10.1 of this library was found to be prone to code injection if used to evaluate untrusted input. The eval function in the Math.js module, not to be confused with the JavaScript eval function we discussed previously, can easily evaluate simple expressions. One of the interesting features is that it allows the use of mathematical functions such as square and cube. The vulnerability is that the library allows creating new functions on the fly via the function constructor. Getting access to it is possible using the .constructor property. We can inject the code calling the function constructor obtained that way, passing malicious code and immediately invoking the resulting function. In this example, our injected code reads environment variables using the env property of the global process object.

Summary

In this module, we discussed the risks that come from passing untrusted input data to the JavaScript engine. We demonstrated how that might lead to the execution of the code provided by the attacker. We discussed the potential impact of such attacks, from denial of service through modification of application code up to complete server takeover using web shells. We identified four unsafe functions that may allow for code injection attacks. The most popular is eval. The function constructor serves a similar purpose, but is used under different circumstances. Two browser functions, setTimeout and setInterval, have unsafe variants that should not be used. We also discussed how a code injection vulnerability was introduced through an open source library and that such libraries should only be used with validated input data and audited for use of unsafe functions.

Defending against Prototype Pollution

Overview

Welcome back to the JavaScript Security: Best Practices Pluralsight course. My name is Marcin Hoppe, and I will teach you how to become a better developer by writing more secure JavaScript. In the previous module, we learned about code injection vulnerabilities. We learned that dynamic execution of code supplied in user input may lead to attacks with significant impact. In this module, we will take a look at security implications of yet another unique JavaScript feature, prototypes. We will learn that allowing modification of the prototype chain may change the behavior of our code in ways that are hard to predict. This type of vulnerability is called prototype pollution. The task of the object model is a bit



unusual. Each object has a chain of 0 or more prototype objects. The object inherits every property and method in the prototype chain. It is the core JavaScript inheritance mechanism. The prototype chain is, by default, mutable and can be modified at runtime. This is rarely desirable, but there's another consequence of the dynamic nature of JavaScript. In practice, the most common case when this might happen is when parsing untrusted JSON documents into JavaScript objects and using those objects to dynamically access properties using the bracket notation. We will analyze this in great detail throughout the module. We will also explain what the possible impact of prototype pollution attacks in our application can be. We will also demonstrate several techniques that may prevent prototype pollution attacks from occurring. Let's see how a prototype pollution attack usually looks like in a typical web application that consists of the client, such as a web browser, and the server. The attacker sends a malicious JSON payload to the target server. The code under attack parses the JSON document and processes the result using vulnerable code. One of the most common cases is merging the parsed JavaScript object with internal application objects. At this point, the prototype chain might have been polluted and the code behavior is not what developers of the applications have originally designed. Before diving into the details, let's review how prototypal inheritance works.

Understanding Prototypes

The majority of mainstream programming languages are object-oriented. One of the core features of object orientation is inheritance. Classes are the most common way to support inheritance. The derived class inherits methods and properties from the base class. The base class may have its own base class. This way, classes form a hierarchy of types. This is how Java, C#, C++, and other popular object-oriented programming languages work. JavaScript uses a different mechanism called prototypes. Each JavaScript object has a prototype that is also an object. The prototype may have its own prototype. That way, prototypes form a chain of objects. One of the most notable differences is that inheritance relationships between classes don't change at runtime. This is not the case for the JavaScript prototype chain. It can change during the execution of the program. Let's dive a little deeper into the prototype chain. Each JavaScript object has a prototype. This is true for all types. Object literals, strings, numbers, functions, and so on. Prototypes form a chain. Null is a JavaScript value to indicate missing data. It is also the value that indicates the end of the prototype chain. The last object in the prototype chain has a prototype equal to null. JavaScript objects have two types of properties, inherited and own. Own properties are the ones that are directly declared when the object is created, or added at runtime to a particular object. Inherited properties are the ones that are available through the prototype chain. They are accessed in exactly the same way as own properties via the dot or bracket notation. One important aspect is



property mutation. When an inherited property is set, only the actual object is modified, never any of the prototypes. It may be counterintuitive when the property you are mutating is an inherited one, and it's worth keeping this detail in mind. Let's take a look at a code sample to better understand how prototypes work. First, we create an object called parent. This object has one property named a with a value of 99. Then, we use the Object.create method to create a child object with parent as the prototype. Then, we read the property a of the child object. The child object does not define a, so JavaScript looks for it in a prototype chain. The prototype of the child object is parent that has the property a, so reading it through the child object returns the expected value, 99. JavaScript uses a special property named proto to return the prototype. This property name works both with the dot and the bracket notation. We will use this special property in the next clip in a prototype pollution example. JavaScript has the class and extends keywords that look exactly like their equivalents in other programming languages, such as Java. They are very convenient to use, but they can also be a little bit misleading. JavaScript always uses prototypal inheritance. Class and extends keywords are just a syntactic sugar to make class creation and setting up the prototype chain easier.

Polluting the Object Prototype

In the last module, we have seen how the simplest attack we could run using a code injection vulnerability was denial of service. The same is true for prototype pollution. Modifying the prototype chain is an easy way to change the internal structure of the code in a way that makes it impossible to execute correctly. One simple way to do it is to modify built-in methods for JavaScript objects, such as toString and valueOf. The for-in loop iterates over all properties of an object, both own and inherited. Adding additional properties to the prototype may force such a loop to process items injected by the attacker. Prototype pollution allows for injection of properties that would normally not be available on certain objects. If security checks and decisions are made based on presence of an object property, then prototype pollution may easily lead to bypass of such checks. Adding properties to query objects may lead to SQL or NoSQL injection vulnerabilities and sensitive data leaks. Combining prototype pollution with other vulnerabilities may sometimes even lead to remote code execution attacks. Let's see how unwanted modification of the prototype chain may lead to bypass of security checks. In this sample code, we will first create a user object. This object represents a regular user and has just one property. The property is called name, and it stores the full name of the user. The attacker may create another object called malicious. This object has a property called isAdmin with a value true. If the application makes security decisions, for example, in access control based on presence and value of such a property, then the ability to inject it to the object representing a user controlled by the attacker is a very attractive target. In our example here, attacker found a way to assign a malicious object to the special



proto property of the user object. This is where the prototype pollution occurs. The user object now inherits the isAdmin property from its injected prototype. Any access control code that grants additional privileges to system administrators based on the presence and value of this property can now be fooled, and if the attacker controls the user that was a victim of this attack, they can escalate their privileges to that of a system administrator.

Finding Prototype Pollution in the Code

At a glance, it looks like the code has to be structured in a very special way to allow for attackers to take advantage of prototype pollution. The canonical pattern that allows for prototype chain modification is property mutation using the bracket notation. If the property key and the value are based on untrusted input data, the attacker can replace the prototype using the special proto property or add a new property to it. Such a construct is often used under the hood in several popular JavaScript idioms and libraries. One of them is deep object merging where the code needs to recursively process each key and value in the source object tree and assign or add it dynamically to the target object. The code that implements this functionality is often generic and works for any shape of the source object and uses the vulnerable pattern under the hood. If the source object can be manipulated by the attacker, it can be used to launch a prototype pollution attack. JavaScript has two ways of creating a shallow copy of an object, the Object.assign method and the spread operator. It does not have any support for making deep copies or cloning of objects. A popular way to do it is to perform a deep recursive merge of the object with the empty target object. Cloning untrusted objects in this way also allows the attacker to tamper with the prototype chain. The last pattern that is often exploitable is writing to properties deep in the object tree based on the path. This path often consists of property names separated by dots. If the path, or any of the components of the path, and the value may be tampered with by the attacker, then prototype pollution is possible. Now, we can take a look at the Wired Brain Coffee ecommerce application to search for one of the code smells. Let's review the saveProfile function once again. This function is the handler for the user profile management form. In the first step, we search the user database to find the user being edited. If the user was found in the database, we clone form fields into a clean, temporary object named updatedUser. Then, we use the Object.assign method to update the user object retrieved from the user database. We do not replace the entire user object because updatedUser does not contain all the required fields, only those that were sent in the user profile management form. Object.assign is a convenient shortcut to update only those properties that are present in the updated user object. It looks like this is exactly like one of the code smells that may lead to prototype pollution. Let's take a look at the merge utility function to see where the prototype chain modification might occur. The merge function uses the for-in loop to iterate over all properties of the source



object. It then uses the bracket notation to add or write the properties based on their name. The value also comes from the source object. In our example, the source object comes from the HTML form and could be easily manipulated by the attacker. Let's see how we might turn this vulnerability into a real attack.

Exploiting the Vulnerability

In this demo, we will abuse the user profile management form to launch a prototype pollution attack. We will use the well-known attack sequence we learned in the last two modules. First, we will hijack a legitimate request using browser development tools. Then, we will inject the malicious payload. And finally, we will deliver the payload to the application. The first attack we're going to launch is a denial of service where we will overwrite one of the built-in object methods. This will allow us to see how such a modification could affect even our simple web application. Then, we will use prototype pollution to enforce specific user credentials when they are missing, opening up a way to another attack called session fixation. Let's log in to the application, and let's go to the user profile management screen. Notice there is just one editable field. Let's type in a new dummy address to see how this information is sent to the server. Looks like the email and the new address are sent in a JSON document. On the server side, this document is parsed, and the parsed object is cloned as we have seen in the previous clip. Let's replace the address property with `proto`. Now, we can inject the new prototype object into the chain. Let's add an object overwriting the built-in object method, `toString`. Let's set it to some dummy value. Now, we can send our modified request. The response seems perfectly normal. Now, let's navigate to the home screen again. Instead of the list of products we expected, we get an error. The error message comes from the Express framework and is caused by the fact that the built-in object `.toString` is no longer a method, but a dummy string. Navigating to the login and user profile management screens gives exactly the same result. A simple modification of the object prototype chain caused a complete denial of service for the entire application. That looks serious, but can we do better? Prototype pollution allows us to add properties to objects when they are not expected. Let's take one more look at the login form handler in the `login.js` file. Notice that getting user credentials breeds them from the request body property. What happens when those fields are missing from the request? Both variables, email and password, have the value `undefined`. This causes the authentication check to fail. Let's get back to the browser and see how that works. Let's open the user profile management form to hijack the vulnerable request again. This time, let's use prototype pollution to add two properties, email and password. Let's set their values to that of our user, `joe@wiredbraincoffee.com`, and `coldbrew`. Yes, that's the secret password. We, once again, send the payload. It seems like the application is working normally. Now, let's log out and get back to the login screen. We can use the browser developer tools to remove both the email and the password fields to



cause both values to be missing. This time we're also logged in. We can see that our user is joe@wiredbraincoffee.com. What happened? Even though the email and password fields were missing from the login request, they were inherited by the request body object in the login function. The values were real, so the login succeeded. If the attacker can somehow trick the victim to send such an empty login request, the victim would be unexpectedly logged in as the attacker. This way, the victim could unknowingly disclose sensitive information to the attacker. Such an attack is usually called session fixation. Now, let's see how we can fix the prototype pollution vulnerability in our application.

Fixing the Code

There are several ways to protect our code from prototype pollution. The most obvious technique that comes to mind is input validation. Validating that untrusted JSON documents do not contain unexpected properties, such as `proto`, should come a long way to protect us from this class of attacks. Input validation can be hard and mistakes can occur. There are several JavaScript features and idioms that can provide additional layers of security. JavaScript allows objects to be frozen. This prevents objects from modification. No new properties can be added to a frozen object, and the values of existing properties cannot be changed. The `Object.freeze` method called on the object prototype can be an effective way to protect against prototype pollution. You should watch out for older JavaScript libraries that extended the object prototype because freezing it can prevent those libraries from working correctly. Some objects do not need to inherit any properties or methods. In such cases, we can minimize the risk of attack if those objects were created without a prototype. This may be a good choice for objects that are merged with untrusted input. Objects without the prototype can be created by calling the `Object.create` method with the first parameter equal to `null`. Using objects to store key value pairs is a popular JavaScript idiom. Another way to achieve the same goal is to use a map. In addition to preventing prototype pollution, it also offers a richer API and increased performance. Now, let's take a look at how we might apply those techniques to our own code. The root cause of the vulnerability is in the `merge` function in the `utils.js` file. We allow any property name from the source object to be assigned to the target object, even the special `proto` property. With the input validation approach, we would have to check if the property is not equal to `proto`. This is easy to do in our simple application, but could be a challenge in larger systems. Let's try to apply the prototype freezing technique. The `app.js` file is the entry point to our application and is executed first. Let's add some code right after the express application has been initialized. First, we need to get the object prototype. We can use the special `proto` property or the `prototype` property of the object constructor function, then we can freeze this prototype by calling the `Object.freeze` method. Let's get back to the browser and let's repeat the attack sequence. First, let's send the



legitimate request to update the address. Then, we edit the request and inject the prototype pollution payload. Finally, we can send the payload to the application. Now we can successfully navigate to any other page of the application. It looks like the attack has been stopped. What happened was that the property modification in the merge function was ignored because the object prototype was frozen.

Introducing Prototype Pollution through 3rd Party Libraries

The JavaScript Standard Library is very small. It provides basic functionality such as mathematical operations, string operations, regular expressions, and a few basic data structures. This resulted in a rich ecosystem of JavaScript libraries and frameworks developed over the years. Some are very comprehensive and provide very rich APIs. Some are small and focused on doing one task well. Many of them implement merging, cloning, and extending objects. We know that those idioms, if not implemented in a robust manner, may contain security vulnerabilities, including prototype pollution. One way to prevent it would be to never use such libraries with untrusted data. Unfortunately, this would greatly reduce their usefulness. It is not a practical approach. Over the years, we have seen the open source security research community discover and publish a number of vulnerability reports in several popular libraries, such as jQuery, a very popular library that makes working with the Document Object Model of the browser much easier, Lodash, a modern utility library augmenting the JavaScript Standard Library with many useful functions, and hapi, one of the most popular web frameworks for Node.js. Using third-party libraries in our own code requires ongoing maintenance, staying up to date on security reports, and updating libraries to safe versions as soon as possible. Now let's take a look how the same attack can occur when using a well-known, third-party library. Let's go to the profile.js file and let's import the Lodash library. The convention is to reference Lodash functions through an underscore variable. Now, let's replace the use of our hand-written merge function with a mature version from Lodash. Now, let's see if we can still run the same prototype pollution attack. I hope you know the attack sequence by now. Intercept the request, inject the payload, and deliver the payload to the application. I did not close the browser window from the previous session, so I can just find a previous malicious request and just send it again. Now we can navigate to any other screen in the application, and notice the familiar error message. The attack succeeded because behind the scenes I used a vulnerable version of the Lodash merge function, 4.17.4. Luckily for us, there is an updated version of this library. In the next module, we will learn how to find out which versions have security flaws and which versions are safe to use.



Summary

In this module, we learned that JavaScript has a unique inheritance model based on prototypes and that this model can be abused by attackers. Modification of the prototype chain may lead to unexpected changes in how our code works. We saw how the basic attack technique is writing to a special property, `__proto__`, that allows easy access to object prototype. We can use it to add unexpected properties to the prototype or replace the prototype altogether. We also discussed several techniques to mitigate prototype pollution attacks. As always, solid input validation and preventing unexpected properties in untrusted JSON documents should be our first line of defense. Using `Map` instead of objects to store key value pairs is another useful technique that helps prevent prototype pollution. The most robust, however, is either freezing the prototype to prevent malicious modifications or creating objects without the prototype. Both techniques fix the root cause of the prototype pollution vulnerability.

Testing Your Code

Overview

Welcome back to the JavaScript Security: Best Practices course on Pluralsight. I am Marcin Hoppe, and I will teach you how to write more secure JavaScript code. In the previous modules, we learned about several types of security vulnerabilities that are unique to JavaScript code. First, we saw how quirks of the dynamic type system may lead to bypass of security checks. Then, we described how dynamic code execution enabled by unsafe functions, such as `eval`, can lead to application and server takeover. Finally, we demonstrated how abusing the prototypal inheritance mechanism can enable a wide variety of attacks. For every class of vulnerabilities, we covered several mitigation techniques we can apply to fix them. In this module, we will focus on how to use automated testing tools to detect some of those security vulnerabilities in our own code. Automated testing is an accepted best practice in software development. In its simplest form, automated tests verify correctness of our code. They take skill and effort to write, but they are an effective way of increasing the quality of our code and preventing bugs. Tests can be used to verify other aspects of code quality, performance, scalability, robustness, as well as security. We will see how we can analyze the code for use of unsafe functions. This will help us prevent code injection attacks. Prototype pollution vulnerabilities are not easy to discover using static code analysis, and the technique we will use to prevent code injection would not work. We will use another technique, unit testing, to detect prototype pollution at runtime. We have seen how vulnerabilities can be introduced by third-party code we do not own. We will take a look at how to analyze external libraries for vulnerabilities. The Wired Brain Coffee ecommerce application had



several vulnerabilities. We found them by manually reviewing the code. Automated security tests allow us to automate the process of detecting security vulnerabilities in the code. Tests will probably never be as thorough as an experienced code reviewer, but they can simplify the process. They may even be your only chance if you don't have access to engineers with security expertise. Once the code has been analyzed, we can fix the identified vulnerabilities. How do we make sure they are not introduced again by new developers or that similar bugs are not added in new code? Running automated security tests on every code change can help a lot. Security testing at scale is a challenge. Automated security tests can help speed up the process for large code bases. They can also help achieve consistency when multiple developers, or even multiple teams of developers, are working on a single project. Automated tests can run really fast and can deliver just-in-time feedback about the code that is currently being worked on.

Security Testing Techniques

Security testing can be really powerful, but there are so many types of security flaws that no single testing technique can discover all of them. The security industry has given us many great testing tools and approaches. We can classify them in three broad categories. The first category is SAST, or static application security testing. This technique is focused on analysis of code and binaries to detect known bad patterns. The second category is DAST, or dynamic application security testing. These type of security tests exercise a running application and look for suspicious responses to a variety of payloads. The third category is IAST, or interactive application security testing. This approach requires that application code is instrumented to detect malicious code behavior within the application. When the application is used or tested, the agent reports detected attacks. IAST is a new and exciting technique, but it is not nearly as popular as the first two. We will not be covering IAST in this module. Let's take a look at pros and cons of SAST and DAST applied to JavaScript code. Both SAST and DAST attempt to identify security vulnerabilities in JavaScript applications. SAST requires access to the source code to analyze. This makes it applicable to a broad range of applications, but it is not a good fit for proprietary applications where we don't have access to the internals. Working at the source code level allows static analysis tools to be very precise in locating the vulnerabilities. The tools will often pinpoint specific lines of code that need to be fixed. DAST, on the other hand, requires a running application to analyze. This makes it particularly useful for web applications. Unfortunately, it is harder to use for other types of programs, for example, command line utilities. Security vulnerabilities reported by DAST tools require manual analysis to find the root cause at the code level. Static analysis testing tools have multiple rules that describe bad coding patterns. Dynamic tools operate on sets of malicious payloads and analyze application output for predetermined signs of a successful attack. The difference has significant impact



on how safe those tools actually are to use. SAST tools only read code, and there is no negative effect that they can have on live systems and customer data. DAST tools, on the other hand, send payloads to our running system. This may accidentally cause data or availability loss. That's why DAST tools should not be run against production environments if possible. Static analysis can be performed by a wide variety of tools, compilers, linters, and dedicated scanners. Dynamic analysis can be performed by our own automated tests, as well as standalone scanners. Both tools rarely report the same type of issues, so you most likely need to have both in your toolbox.

Finding Unsafe Code Using ESLint

Linters are a fast and useful way to find bugs and simple coding mistakes. ESLint is the most popular linting tool for JavaScript. Linting and the code execution start with the same step, parsing the code and building an abstract syntax tree, or AST for short. The actual analysis is performed on the AST and not on the raw code. ESLint has a modular architecture. The checks are not built directly into the tool itself, but implemented as plugins. This allowed the ESLint community to work on many useful plugins covering things such as coding style, enforcing best practices, and even simple security checks. One of the goals of linters is performance. They run quickly enough to be easily integrated into code editors, IDEs, and build scripts. In this demo, we will learn the basics of ESLint. We will install the tool and perform basic configuration steps. The core part of setting ESLint up is choosing which rules to run. We will focus on the rules that detect usage of unsafe functions, eval and the function constructor. We will run ESLint on the original version of the Wired Brain Coffee ecommerce application and demonstrate how it allows us to detect a potential code injection attack. The first step to run ESLint is to install it. The simplest way to do it is to use the npm package manager and its npm install command. Remember about the saveDev switch to indicate that ESLint is a development dependency and is not needed to run the code. The installation may take some time, so be patient. The next step is to initialize ESLint configuration for our project using the eslint init command. The configurator will ask us a few questions. We will use it to check syntax and find problems. Our project is using the CommonJS module system, and we are not using any of the currently popular front-end frameworks. The code we want to lint runs on Node.js. Let's store our configuration in JSON. The last step is installation of the latest set of rules. This may also take a little while. ESLint configuration is stored in a file called .eslintrc.json. The extends property turns on the recommended set of rules. Let's remove it for the time being to start with a clean slate. The file has a rules section where we can specify which checks we want to run. No-eval catches all direct calls to eval. No-implied-eval flags uses of unsafe variants of setTimeout and setInterval. No-new-func prevents the code from using the function constructor. All rules will generate an error. This is important if you want to break the build if any of the unsafe functions are



used. Now, let's run ESLint to analyze all the JavaScript files in the lib folder. You can see that the tool raised one error and pointed directly at line 11 in the login.js file. That's right, here's our call to the unsafe eval function.

Detecting Prototype Pollution with Unit Tests

We already discussed the benefits of test automation. Now, we are going to take a more detailed look on how to use unit tests to detect prototype pollution. Using unit tests to test our code for security flaws is both reliable and repeatable. Automated security tests are unambiguous, and the result does not rely on the skills and knowledge of the person performing manual code review. Commercial DAST scanners work the best for applications available through the network, such as web applications. It is difficult to use them to test those parts of our code that are not easily reachable. Using unit tests instead allows for much easier delivery of malicious payloads to the application. Unit tests also make it easier to inspect the state of the program after a malicious payload has been processed. Using unit tests, we may be able to catch errors coming from automatic conversions and loose comparisons. We may also easily check if code injection attempts were successful. We can also inspect inherited properties of objects on our test to detect prototype pollution. In this demo, we will create a unit test that detects the prototype pollution vulnerability in the merge utility function. We will start by introducing and setting up Mocha, a very popular JavaScript test runner. We could achieve the same goal with any unit test framework, or even without one. Mocha is simple and popular and will allow us to get rid of the boilerplate and focus on the essence of the test. Our attempt at detecting prototype pollution will consist of two phases. First, we design a malicious payload that will be very similar to the one we used in module 4. We will inject the property to the object prototype. The second step is verification if the injected property is available as an inherited property on another unrelated object. First, we need to install Mocha using the npm package manager. Remember to pass the saveDev switch to the npm install command to indicate this is a development dependency. As with ESLint, the package installation may take a while. The next step is to create a new JavaScript file for tests. There are several different conventions to organize test code, but we will keep it simple. Let's create a single file called utils.test.js in the lib folder. Let's import the function we want to test, the merge function from utils.js. We will also need to import the assert module to implement checks in our unit tests. Mocha test suites are defined using the describe function. Individual tests within them are defined using the it function. The first parameter of both functions is the name, the second parameter is a function with the body of the suite or the test. First, we use JSON.parse to create the object that contains the malicious payload. The JSON document uses the special proto property to create a new property called injected with a value equal to 0. Next, we called the merge function with an empty object as the target and the payload as the source. We



are almost done. Let's create an entirely new object. Now, let's check if this new object, unrelated to the payload, has the inherited property called injected. If this check passes, the function is not vulnerable. If the check fails, it is prone to prototype pollution. Let's run the test by passing the path to the `utils.test.js` file to the Mocha test runner. We can see that the test failed, which means the merge function is vulnerable to prototype pollution. We were expecting `own.injected` expression to return value undefined, but we got `0` instead. This is because the property is now inherited from the modified prototype chain. We managed to write a unit test that successfully detected prototype pollution.

Popular Security Testing Tools for JavaScript

There are plenty of security testing tools. Some support JavaScript better than others. Let's take a quick tour of free and popular commercial tools that can be used for JavaScript security testing. SAST is a very crowded market. There are both open source tools, as well as established commercial vendors. We have already mentioned ESLint. It is very popular among JavaScript programmers, and using it to analyze code for simple security vulnerabilities and coding antipatterns is an easy way to start. GitHub is a very popular development platform for both open source and commercial projects. It offers a static analysis scanner as a part of its advanced security offering, including support for JavaScript. Similar capabilities are available for free for open source projects through the LGTM service, also operated by GitHub. Semgrep is an open source, lightweight static analysis tool that can be run locally from the command line. It supports several different programming languages, including JavaScript. OWASP Zed Attack Proxy, or ZAP for short, is the most popular open source DAST scanner. It can be used to perform dynamic analysis of JavaScript applications. DAST is also a very crowded market, and there are many alternatives to OWASP ZAP. JavaScript has a very rich third-party package ecosystem. Npm is the most popular package manager for JavaScript. Managing dependencies is an important measure to prevent security vulnerabilities introduced through external libraries. Npm audit is a tool built directly into the package manager command line interface. It scans project dependencies for vulnerabilities. Retire.js is a vulnerability database and scanner for JavaScript. It can be called from the command line, used as a browser extension, or as a DAST plugin. Dependency-Track is a comprehensive software composition analysis tool that helps manage the use of open source components in complex systems. It integrates with several vulnerability databases and has good support for JavaScript packages from npm. Dependency-Track is itself an open source project developed by OWASP. Another open source security tool is Snyk. It is a proprietary tool, but it can be used for free or open source projects. Let's take a look at how we can detect a vulnerable third-party library in our own code. In this short demo, we will analyze the dependencies of the Wired Brain Coffee application. The back end uses several open source libraries for Node.js,



such as Express, and uses npm as the package manager. We will use npm audit to scan the list of dependencies of our application to detect a vulnerable Lodash version, and we will use npm to update the library to a safe version. Let's start with examining the list of libraries that our application depends on. We can do this using the npm list command. You can see that our simple application uses many different libraries and versions. It is possible that one or more of those libraries contain security vulnerabilities. We can check this using the npm audit command. You can see that the version of Lodash we use contains eight different security vulnerabilities. Five of them are prototype pollution bugs. We can use npm install to upgrade to the latest version of Lodash. Running npm audit confirms that we have updated Lodash from 4.17.4 to 14.17.19 and that this version has no known security vulnerabilities

Summary

In this module, we learned about the most popular security testing techniques. SAST, or static application security testing, is a technique that focuses on source code analysis. SAST tools look for known bad patterns that can introduce security vulnerabilities. DAST, or dynamic application security testing, focuses on testing the running application, usually over the network. DAST tools send specially prepared payloads to the application and analyze responses to look for signs of successful attacks. IAST, or interactive application security testing, is a combination of the previous two approaches. This technique is very promising, but it is not very popular yet. We demonstrated how to use automated SAST and DAST tests to discover vulnerabilities explained in previous modules. We demonstrated how to use ESLint to prevent a code injection vulnerability. We also saw how to use unit tests to detect prototype pollution in the merge utility function. We also learned how to use npm audit to detect vulnerabilities introduced through third-party code.